

Operating Systems

Assignment 2

Sambit Sahoo (22277)

*My username is **sambit** (You can find it in all screenshots.)*

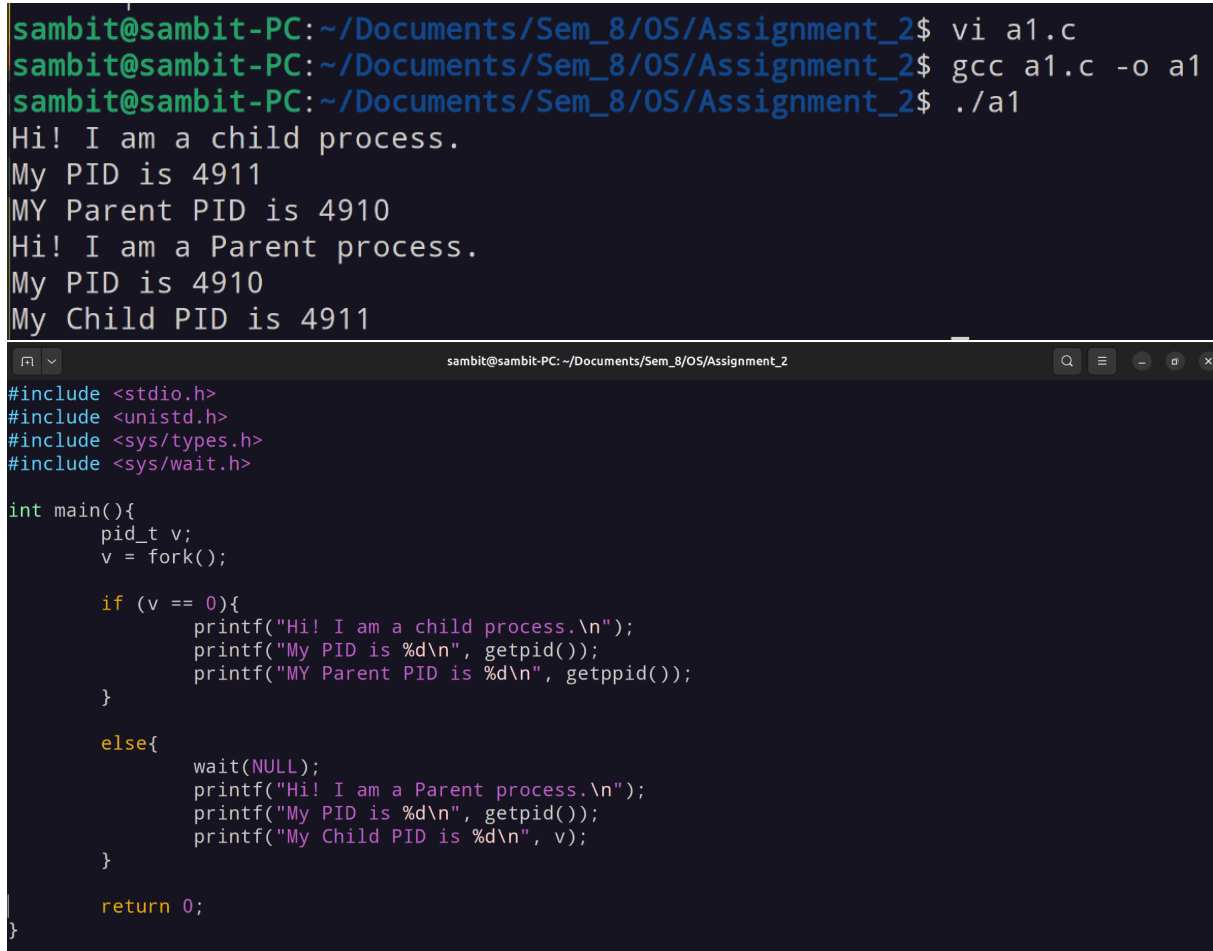
Q1 - Implement a C/C++ program that represents a parent-child relationship between two processes. Parent and child processes perform print tasks as follows:

1. Parent:

- (a) Hi! I am a parent process.
- (b) My PID is <PID>.
- (c) My Child PID is <PID>.

2. Child:

- (a) Hi! I am a child process.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.



```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ vi a1.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ gcc a1.c -o a1
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ ./a1
Hi! I am a child process.
My PID is 4911
MY Parent PID is 4910
Hi! I am a Parent process.
My PID is 4910
My Child PID is 4911
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(){
    pid_t v;
    v = fork();

    if (v == 0){
        printf("Hi! I am a child process.\n");
        printf("My PID is %d\n", getpid());
        printf("MY Parent PID is %d\n", getppid());
    }

    else{
        wait(NULL);
        printf("Hi! I am a Parent process.\n");
        printf("My PID is %d\n", getpid());
        printf("My Child PID is %d\n", v);
    }

    return 0;
}
```

Q2 - Implement a C/C++ program with parent-child process relations that accepts N integers (Note that N can be anything based on user input) as input.

The parent process sorts these integers using **Merge Sort**, whereas the child process sorts these integers using **Quick Sort** and prints the respective results in the following format:

1. Parent:

- (a) Hi! I am a parent process.
- (b) My PID is <PID>.
- (c) Merge Sort Result: <Sorted Result>.

2. Child:

- (a) Hi! I am a child process.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.
- (d) Quick Sort Result: <Sorted Result>.

```
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ vi a2.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ gcc a2.c -o a2
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ ./a2
Sambit Sahoo      22277
Enter number of integers: 10
Enter 10 elements:
19
68
53
37
91
2
26
33
84
64
Hi! I am a child process.
My PID is 13913
MY Parent PID is 13888
Quick Sort Result:2 19 26 33 37 53 64 68 84 91
Hi! I am a Parent process.
My PID is 13888
Merge Sort Result:2 19 26 33 37 53 64 68 84 91
```

```

sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int partition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low + 1;
    int j = high;
    int temp;

    while (1) {
        while (i <= high && arr[i] <= pivot)
            i++;

        while (arr[j] > pivot)
            j--;

        if (i >= j)
            break;

        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    temp = arr[low];
    arr[low] = arr[j];
    arr[j] = temp;

    return j;
}

```

```

sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2

void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[high + 1];

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];

    for (int x = low; x <= high; x++)
        arr[x] = temp[x];
}

void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

```

```

sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2

}

int main() {
    int n;
    printf("Sambit Sahoo\t 22277\n");

    printf("Enter number of integers: ");
    scanf("%d", &n);

    int arr[n];
    int arr_copy[n];

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++){
        scanf("%d", &arr[i]);
        arr_copy[i] = arr[i];
    }

    pid_t v;
    v = fork();

    if (v == 0){
        printf("Hi! I am a child process.\n");
        printf("My PID is %d\n", getpid());
        printf("My Parent PID is %d\n", getppid());
        printf("Quick Sort Result:");
        quickSort(arr_copy, 0, n - 1);
        for (int i = 0; i < n; i++)
            printf("%d ", arr_copy[i]);

        printf("\n");
    }

    else{
        wait(NULL);
        printf("Hi! I am a Parent process.\n");
        printf("My PID is %d\n", getpid());
        printf("Merge Sort Result:");
        mergeSort(arr, 0, n-1);
        for (int i = 0; i < n; i++)
            printf("%d ", arr[i]);

        printf("\n");
    }
}

```

Q3 - Write a C program with a parent-child relationship and make the child process orphan by terminating the parent process earlier than the child.

The parent process performs the task of calculating the factorial of n numbers (User Input). Where as, the task performed by the child process is to generate m (User-based input) Fibonacci numbers. Note that $m \gg n$. The output format is as follows:

1. Parent:

- (a) Hi! I am a parent process.
- (b) My PID is <PID>.
- (c) Factorial for number 5: 120.
- (d) I am terminated earlier than my child.

2. Child:

- (a) Hi! I am a child process.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.
- (d) Fibonacci numbers for 10 numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ vi a3.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ gcc a3.c -o a3
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ echo "10 5" | ./a3 > a3_output.txt
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ cat a3_output.txt
Hi! I am a parent process.
My PID is 6493
Factorial for number 5: 120
I am terminated earlier than my child.
Hi! I am a child process.
My PID is 6494
MY Parent PID is 2016
Fibonacci numbers for 10 numbers:
0,1,1,2,3,5,8,13,21,34.
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int fib(int n){
    if (n == 1){
        return 0;
    }
    else if (n == 2){
        return 1;
    }
    else{
        return fib(n-1)+fib(n-2);
    }
}

int fac(int n){
    if (n == 0 || n == 1){
        return 1;
    }
    else{
        return n * fac(n-1);
    }
}
```

```
int main()
{
    int n=0,m=0;
    scanf("%d",&m);
    scanf("%d",&n);
    pid_t v;
    v=fork();
    if (v == 0){
        printf("Hi! I am a child process.\n");
        printf("My PID is %d\n",getpid());
        printf("MY Parent PID is %d\n", getppid());
        printf("Fibonacci numbers for %d numbers:\n",m);
        for (int i = 1; i <= m-1; i++){
            printf("%d,",fib(i));
        }
        printf("%d.\n",fib(m));
    }
    else{
        //wait(NULL);
        printf("Hi! I am a parent process.\n");
        printf("My PID is %d\n", getpid());
        printf("Factorial for number %d: %d\n",n, fac(n));
        printf("I am terminated earlier than my child.\n");
    }
    return 0;
}
```

Q4 - Write a C program showing the parent-child relationship with a tree structure as shown in Figure . The processes print messages to verify the parent-child relationship. The sample output format is as shown below:

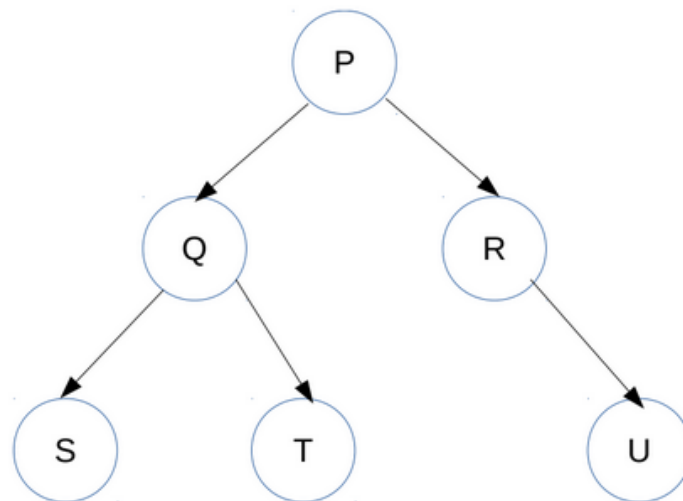


Figure 2: Parent Child Tree Relationship

1. **P:**
 - (a) Hi! I am a root process.
 - (b) My PID is <PID>.
2. **Q:**
 - (a) Hi! I am a child process Q of Parent P.
 - (b) My PID is <PID>.
 - (c) My Parent PID is <PID>.
3. **R:**

- (a) Hi! I am a child process R of Parent P.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.

4. **S:**

- (a) Hi! I am a child process S of Parent Q.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.
- (d) My Grand Parent PID is <PID>. (Optional)

5. **T:**

- (a) Hi! I am a child process T of Parent Q.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.
- (d) My Grand Parent PID is <PID>. (Optional)

6. **U:**

- (a) Hi! I am a child process U of Parent R.
- (b) My PID is <PID>.
- (c) My Parent PID is <PID>.
- (d) My Grand Parent PID is <PID>. (Optional)

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ gcc a4.c -o a4
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ ./a4 > a4_output.txt
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_2$ cat a4_output.txt
U:
Hi! I am a child process U of Parent R.
My PID is 9306.
MY Parent PID is 9304.
My Grand Parent PID is 9303.
T:
Hi! I am a child process T of Parent Q.
My PID is 9308.
MY Parent PID is 9305.
My Grand Parent PID is 9303.
S:
Hi! I am a child process S of Parent Q.
My PID is 9307.
MY Parent PID is 9305.
My Grand Parent PID is 9303.
R:
Hi! I am a child process R of Parent P.
My PID is 9304.
MY Parent PID is 9303
Q:
Hi! I am a child process Q of Parent P.
My PID is 9305
MY Parent PID is 9303
P:
Hi! I am a root process.
My PID is 9303
```

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2
#include<stdio.h>
#include<sys/wait.h>
#include<sys/types.h>
#include<unistd.h>

int main(){
    pid_t p0, p1, p2;
    p0 = getpid();
    p1 = fork();
    p2 = fork();

    if(p1 > 0 && p2 > 0){
        printf("P:\n");
        printf("Hi! I am a root process.\n");
        printf("My PID is %d\n", getpid());
        wait(NULL);
    }

    else if (p1 == 0 && p2 > 0){
        printf("R:\n");
        printf("Hi! I am a child process R of Parent P.\n");
        printf("My PID is %d\n", getpid());
        printf("MY Parent PID is %d\n", getppid());
        wait(NULL);
    }

    else if (p1 == 0 && p2 == 0){
        printf("U:\n");
        printf("Hi! I am a child process U of Parent R.\n");
        printf("My PID is %d\n", getpid());
        printf("MY Parent PID is %d\n", getppid());
    }
}
```

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_2
    printf("My Grand Parent PID is %d.\n",p0);
}

else{
    pid_t p3, p4;
    p3 = fork();
    if (p3 == 0){
        printf("S:\n");
        printf("Hi! I am a child process S of Parent Q.\n");
        printf("My PID is %d.\n",getpid());
        printf("MY Parent PID is %d.\n", getppid());
        printf("My Grand Parent PID is %d.\n",p0 );
    }

    else{
        p4 = fork();
        if (p4 == 0){
            printf("T:\n");
            printf("Hi! I am a child process T of Parent Q.\n");
            printf("My PID is %d.\n", getpid());
            printf("MY Parent PID is %d.\n", getppid());
            printf("My Grand Parent PID is %d.\n", p0);
        }

        else{
            printf("Q:\n");
            printf("Hi! I am a child process Q of Parent P.\n");
            printf("My PID is %d\n", getpid());
            printf("MY Parent PID is %d\n", getppid());
            wait(NULL);
        }
    }
}
}
```